

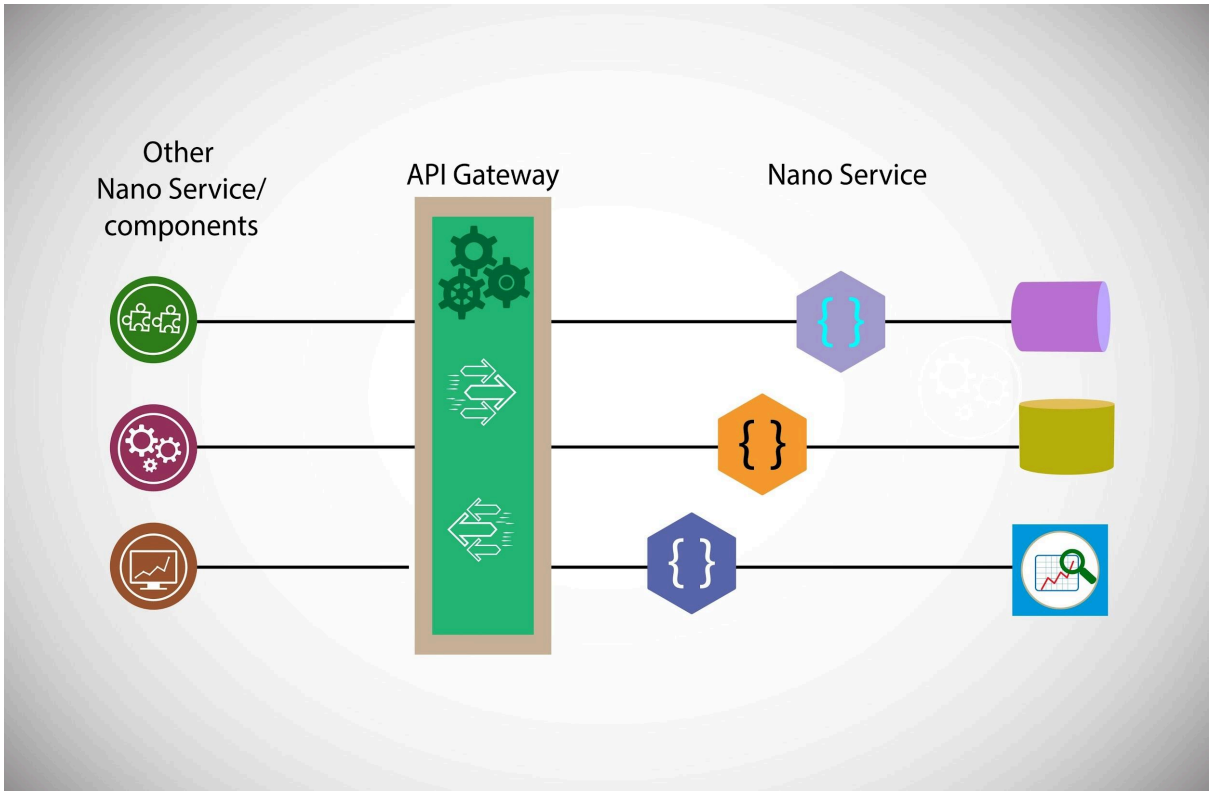
Banking System Simulator

(Microservices Architecture)

This project simulates a basic banking platform refactored into a microservices architecture using **Spring Boot**, **Spring Cloud**, and **MongoDB**. The focus is on modularity, inter-service communication via **Feign Client**, **Service Discovery (Eureka)**, and **Fault Tolerance (Resilience4j Circuit Breaker)**.

1. 🏗️ Architecture Diagram (Conceptual)

The system consists of five independent services orchestrated via the API Gateway and Eureka.



Shutterstock

Service	Port	Database	Description
Eureka Server	8761	—	Central Service Discovery registry.
API Gateway	8080	—	Routes all external traffic to the correct service.

Account Service	8081	<code>accounts_db</code> (MongoDB)	Manages account creation and balance updates.
Transaction Service	8082	<code>transactions_db</code> (MongoDB)	Handles deposits and transfers. Orchestrates balance updates (via Feign) and notifications. Implements Resilience4j .
Notification Service	8083	—	Receives transaction events and simulates sending notifications.

Export to Sheets

2. Project Setup

Prerequisites

- **Java 17+**
- **Apache Maven**
- **MongoDB:** A local MongoDB instance running on the default port **27017**.
- **Docker (Optional):** For running containers via Dockerfiles.

Local Setup Steps

1. **Clone the Repository (or setup the structure):** Ensure you have the five service directories (`eureka-server`, `api-gateway`, `account-service`, `transaction-service`, `notification-service`).

Verify Configuration: Confirm that the `application.properties` file in the `eureka-server` directory contains the self-referencing fix to prevent dashboard errors:

```
Properties
eureka.client.service-url.defaultZone=http://${eureka.instance.hostname}:${server.port}/eureka/
eureka.instance.hostname=localhost
```

- 2.

Package All Services: Open the terminal in the root directory (`banking-microservices`) and compile all projects:

```
Bash
mvn clean install
```

- 3.

3. Run Instructions

You must start the services in a specific order. Open **five separate terminal windows**.

Startup Sequence

Step	Service	Command	Status Check
1	Eureka Server	<code>cd eureka-server && mvn spring-boot:run</code>	Wait until fully started.
2	Account Service	<code>cd account-service && mvn spring-boot:run</code>	Registers with Eureka.
3	Notification Service	<code>cd notification-service && mvn spring-boot:run</code>	Registers with Eureka.
4	Transaction Service	<code>cd transaction-service && mvn spring-boot:run</code>	Registers with Eureka.
5	API Gateway	<code>cd api-gateway && mvn spring-boot:run</code>	Registers with Eureka.

Export to Sheets

Verification

Check the Eureka Dashboard: <http://localhost:8761/>

All three business services ([ACCOUNT-SERVICE](#), [TRANSACTION-SERVICE](#), [NOTIFICATION-SERVICE](#)) should show as **UP**.

4. Testing the Flow (via API Gateway: Port 8080)

Use Postman or cURL to execute the following requests.

4.1. Create Accounts

Purpose: Test routing to **Account Service** via Gateway.

- **Endpoint:** [POST http://localhost:8080/api/accounts](http://localhost:8080/api/accounts)

Body (JSON):

JSON

```
{"accountNumber": "10001", "holderName": "Alice", "balance": 1000.0}
```

```
{"accountNumber": "10002", "holderName": "Bob", "balance": 500.0}
```

-

4.2. Test Deposit and Notifications

Purpose: Test **Transaction Service** orchestration (Transaction → Account → Notification).

- **Endpoint:** `POST http://localhost:8080/api/transactions/deposit`

Body (JSON):

JSON

```
{"accountNumber": "10001", "amount": 200.0}
```

-
- **Expected Behavior:**
 - `account-service` balance for **10001** changes to **1200.0**.
 - `notification-service` console prints a successful deposit message.

4.3. Test Transfer

Purpose: Test complex transaction logic involving two atomic balance updates.

- **Endpoint:** `POST http://localhost:8080/api/transactions/transfer`

Body (JSON):

JSON

```
{"sourceAccount": "10001", "destinationAccount": "10002", "amount": 100.0}
```

-
- **Expected Balances:**
 - Account **10001**: $1200 - 100 = 1100.0$
 - Account **10002**: $500 + 100 = 600.0$

4.4. Test Fault Tolerance (Resilience4j)

Purpose: Verify the **Circuit Breaker** works when a downstream service fails.

1. **Stop the Account Service** (Close terminal window 2).
2. **Repeat the Deposit test (4.2).**
3. **Expected Behavior:**
 - The `transaction-service` terminal immediately logs the **CIRCUIT BREAKER OPEN/FAILURE** message.
 - The transaction status saved to `transactions_db` is **FAILED**.
 - The system remains up, demonstrating **fault isolation**.
4. **Restart the Account Service** before continuing.